

EJERCICIO DE REPASO DE USO DE FUNCIONES GENERICAS PLANTILLA DE FUNCIONES (TEMA 2)

Dadas las siguientes clases:

```
#include <iostream>
#include <cstdlib>

using namespace std;

class Fecha {
    int dia, mes, anio;
public:
    Fecha(int d=1, int m=1, int a=2000) { dia=d; mes=m; anio=a; }
    int getDia() const { return dia; }
    int getMes() const { return mes; }
    int getAnio() const { return anio; }
    static int esMayor(Fecha f1, Fecha f2) {
        return ((f1.anio*10000+f1.mes*100+f1.dia) > (f2.anio*10000+f2.mes*100+f2.dia));
    }
    friend ostream& operator<<(ostream &s, const Fecha &f);
};

ostream& operator<<(ostream &s, const Fecha &f) {
    if (f.dia < 10) s << "0";
    s << f.dia << "/";
    if (f.mes < 10) s << "0";
    s << f.mes << "/" << f.anio;
    return s;
}

class Fraccion {
    int n, d;
public:
    Fraccion(int n=0, int d=1) { this->n=n; this->d=d; }
    int getn() const { return n; }
    int getd() const { return d; }
    friend ostream& operator<<(ostream &s, const Fraccion &f);
};

ostream& operator<<(ostream &s, const Fraccion &f) {
    if (f.d>0) s << f.n << "/" << f.d; //signo siempre en el numerador
    else s << -f.n << "/" << -f.d;
    return s;
}
```

Implementa las funciones necesarias para que el siguiente main() se pueda ejecutar:

```
int main() {
    int mint, tint[] = {2, 4, 6, 9, 8, 1}; //array de 6 enteros
    float mfloat, tfloat[] = {4.5, 7.9, 6.0, 0.9, 1.0, 4.3}; //array de 6 float
    double mdouble, tdouble[] = {4.7, 9, 6.2, 9.9, 0.4, 9.3}; //array de 6 double
    Fecha mFecha, tFecha[] = { Fecha(1,1,2000), Fecha(2,3,1999), Fecha(7,9,2000),
                               Fecha(15,8,2000), Fecha(20,4,2000) }; //array de 5 Fechas
    Fraccion mFraccion, tFraccion[] = { Fraccion(4,5), Fraccion(7,9), Fraccion(5,4),
                                         Fraccion(8,7), Fraccion(2,4) }; //array de 5 Fracciones

    mint = maximo(tint, 6);
    mfloat = maximo(tfloat, 6);
    mdouble = maximo(tdouble, 6);
    mFecha = maximo(tFecha, 5);
    mFraccion = maximo(tFraccion, 5);

    cout << "Los valores maximos de cada tabla son: " << endl;
    cout << mint << endl;
    cout << mfloat << endl;
    cout << mdouble << endl;
    cout << mFecha << endl;
    cout << mFraccion << endl;

    system("PAUSE"); return 0;
}
```

Realiza el ejercicio considerando que:

- a) No se pueden usar funciones genéricas (plantilla de funciones).
- b) Se pueden usar funciones genéricas (plantilla de funciones) y especializaciones de plantillas de funciones.
- c) Sólo se puede usar funciones genéricas, pero no especializaciones (indica que cambios habría que introducir en las clases anteriores para poder hacerlo).

SOLUCION:

a) No se pueden usar funciones genéricas (plantilla de funciones).

Al no poder usar funciones genéricas debemos implementar 5 funciones diferentes, una para cada tipo de datos que se usa en el main()

```
int maximo(int tabla[], int n) {
    int mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (tabla[i]>mayor)
            mayor=tabla[i];
    }
    return mayor;
}

float maximo(float tabla[], int n) {
    float mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (tabla[i]>mayor)
            mayor=tabla[i];
    }
    return mayor;
}

double maximo(double tabla[], int n) {
    double mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (tabla[i]>mayor)
            mayor=tabla[i];
    }
    return mayor;
}

Fecha maximo(Fecha tabla[], int n) {
    Fecha mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (Fecha::esMayor(tabla[i],mayor))
            mayor=tabla[i];
    }
    return mayor;
}

Fraccion maximo(Fraccion tabla[], int n) {
    Fraccion mayor=tabla[0];
    float faux, fmayor=(float)mayor.getn()/mayor.getd();
    for(int i=1; i<n; i++) {
        faux=(float)tabla[i].getn()/tabla[i].getd();
        if (faux > fmayor) {
            mayor=tabla[i];
            fmayor=faux;
        }
    }
    return mayor;
}
```

- b) Se pueden usar funciones genéricas (plantilla de funciones) y especializaciones de plantillas de funciones.

Al poder usar funciones genéricas podemos implementar **1 función genérica** que sirve para los tipos **int, float y double** (ya que el código de los 3 es idéntico y sólo cambia el tipo de los datos usados) y 2 especializaciones de dicha plantilla (**una para el tipo Fecha** y **otra para el tipo Fraccion**, ya que el código es distinto en ellos)

```
template <class T> T maximo(T tabla[], int n) { //función generica (sirve para int, float y double)
    T mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (tabla[i]>mayor)
            mayor=tabla[i];
    }
    return mayor;
}

Fecha maximo(Fecha tabla[], int n) { //especializacion para el tipo Fecha
    Fecha mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (Fecha::esMayor(tabla[i],mayor))
            mayor=tabla[i];
    }
    return mayor;
}

Fraccion maximo(Fraccion tabla[], int n) { //especializacion para el tipo Fraccion
    Fraccion mayor=tabla[0];
    float faux, fmayor=(float)mayor.getn()/mayor.getd();
    for(int i=1; i<n; i++) {
        faux=(float)tabla[i].getn()/tabla[i].getd();
        if (faux > fmayor) {
            mayor=tabla[i];
            fmayor=faux;
        }
    }
    return mayor;
}
```

- c) Sólo se puede usar funciones genéricas, pero no especializaciones (indica que cambios habría que introducir en las clases anteriores para poder hacerlo).

Al no poder usar especializaciones, la función genérica debe servir (aparte de los tipos int, float y double) para los tipos Fecha y Fraccion. Por tanto debemos sobrecargar en ambas clases el operator> para que se puede usar con dichos tipos

```
#include <iostream>
#include <cstdlib>

using namespace std;

class Fecha {
    int dia, mes, anio;
public:
    Fecha(int d=1, int m=1, int a=2000) { dia=d; mes=m; anio=a; }
    int getDia() const { return dia; }
    int getMes() const { return mes; }
    int getAnio() const { return anio; }
    static int esMayor(Fecha f1, Fecha f2) {
        return ((f1.anio*10000+f1.mes*100+f1.dia) > (f2.anio*10000+f2.mes*100+f2.dia));
    }
    bool operator>(Fecha f) { return ((anio*10000+mes*100+dia) > (f.anio*10000+f.mes*100+f.dia)); }
    friend ostream& operator<<(ostream &s, const Fecha &f);
};

ostream& operator<<(ostream &s, const Fecha &f) {
    if (f.dia < 10) s << "0";
    s << f.dia << "/";
    if (f.mes < 10) s << "0";
    s << f.mes << "/" << f.anio;
    return s;
}

class Fraccion {
    int n, d;
public:
    Fraccion(int n=0, int d=1) { this->n=n; this->d=d; }
    int getn() const { return n; }
    int getd() const { return d; }
    bool operator>(Fraccion q) { return (n*q.d>d*q.n); }
    friend ostream& operator<<(ostream &s, const Fraccion &f);
};

ostream& operator<<(ostream &s, const Fraccion &f) {
    if (f.d>0) s << f.n << "/" << f.d; //signo siempre en el numerador
    else s << -f.n << "/" << -f.d;
    return s;
}

template <class T> T maximo(T tabla[], int n) {
    T mayor=tabla[0];
    for(int i=1; i<n; i++) {
        if (tabla[i]>mayor)
            mayor=tabla[i];
    }
    return mayor;
}

int main() {
    int mint, tint[] = {2, 4, 6, 9, 8, 1}; //array de 6 enteros
    float mfloat, tfloat[] = {4.5, 7.9, 6.0, 0.9, 1.0, 4.3}; //array de 6 float
    double mdouble, tdouble[] = {4.7, 9, 6.2, 9.9, 0.4, 9.3}; //array de 6 double
    Fecha mFecha, tFecha[] = { Fecha(1,1,2000), Fecha(2,3,1999), Fecha(7,9,2000),
                               Fecha(15,8,2000), Fecha(20,4,2000) }; //array de 5 Fechas
    Fraccion mFraccion, tFraccion[] = { Fraccion(4,5), Fraccion(7,9), Fraccion(5,4),
                                         Fraccion(8,7), Fraccion(2,4) }; //array de 5 Fracciones

    mint = maximo(tint, 6);
    mfloat = maximo(tfloat, 6);
    mdouble = maximo(tdouble, 6);
    mFecha = maximo(tFecha, 5);
}
```

```
mFraccion = maximo(tFraccion, 5);

cout << "Los valores maximos de cada tabla son: " << endl;
cout << mint << endl;
cout << mfloat << endl;
cout << mdouble << endl;
cout << mFecha << endl;
cout << mFraccion << endl;

system("PAUSE"); return 0;
}
```